

CONTRACT NOTE REWORK

Comparison Audit

Estimate 5 - Technical Walkthrough

~12.4 developer days (incl. prompt iteration)

Background

BRYT suspects that some contract note PDFs may be edited by hand after they are rendered but before they reach the customer, sidestepping the controlled template system. Because the edit happens outside any system, there is currently no way to know whether it is occurring or how often.

Estimate 5 is an audit tool that answers that question. For a batch of contract notes it retrieves the original rendered PDF and the version that was actually emailed to the customer, compares the two with AWS Bedrock, and reports any differences it finds.

This is a developer-operated analytical tool, not a customer- or business-facing product. It runs ad-hoc (for example monthly) over a list of references and produces results that are queried with Athena and delivered to BRYT as a spreadsheet.

How it differs from the other estimates

Estimates 1-4 build the production system. Estimate 5 is a separate investigative tool that runs alongside it. There is no UI and no live integration, just a batch pipeline a developer triggers when an audit is needed.

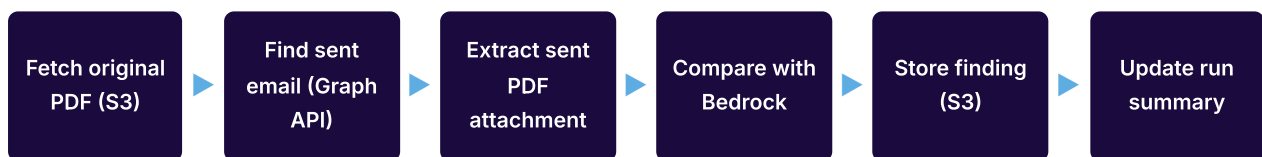
How it works

A developer supplies a list of contract note references (up to 500 per run). A Step Function iterates the list, and for each reference it does three things: finds the original rendered PDF in S3, finds the sent PDF by searching the Outlook mailbox via the Microsoft Graph API, and, if both are available, asks Bedrock to compare them.

The comparison result, identical or not, and if not, what changed, is written to S3 as a finding. A run-level summary tracks how many matched, mismatched, or errored. Once the run finishes, the findings are queried with Athena and turned into a spreadsheet for BRYT.

The two documents come from deliberately different places: the original from the system's own S3 output, and the sent version from the actual email, because the email is the source of truth for what the customer really received.

Per-record comparison flow



If either PDF is missing, an error finding is recorded and the run continues to the next record.

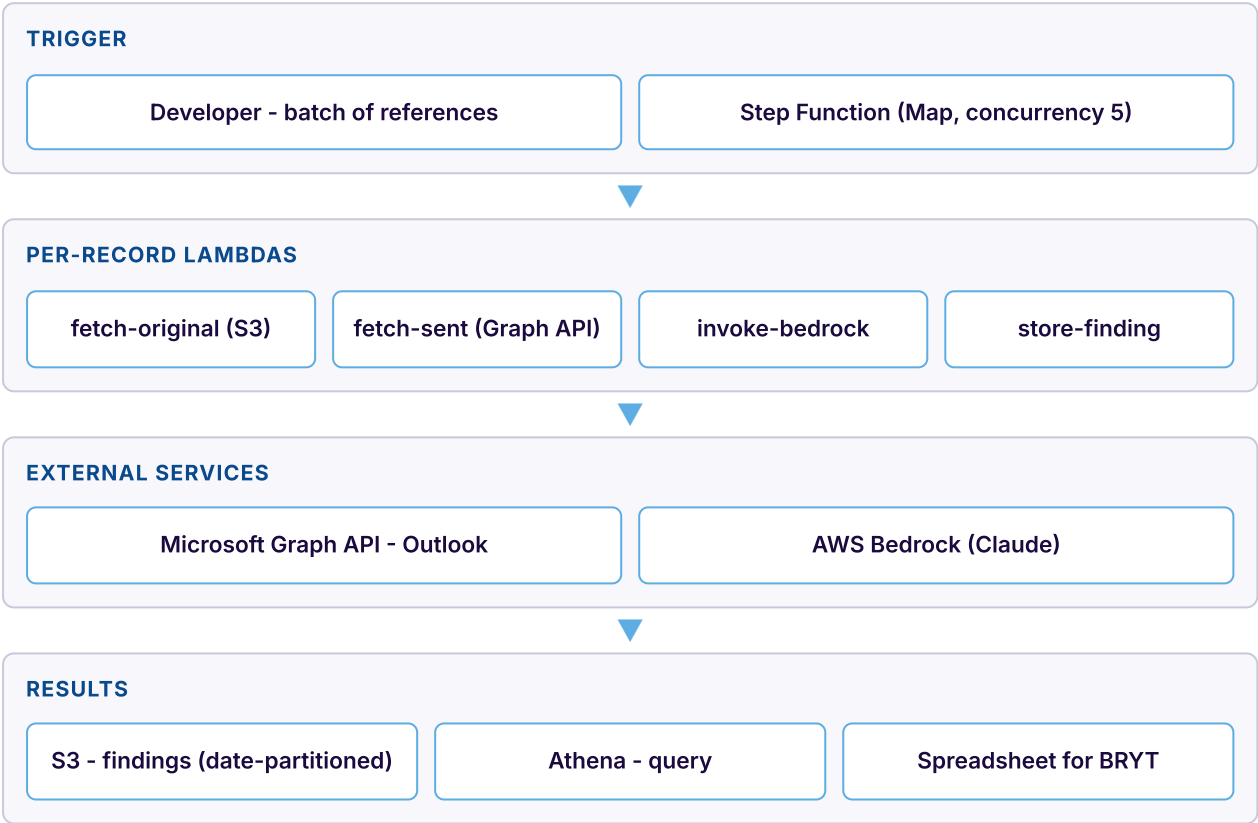
Run lifecycle



Concurrency is capped at 5 to stay within Graph API rate limits.

System architecture

A Step Function orchestrates a handful of small Lambdas, each responsible for one step. The two PDFs are sourced from S3 and Outlook respectively, compared by Bedrock, and the findings land back in S3 where Athena can query them.



A developer-operated batch tool; nothing here runs in the live contract note path.

Key design decisions

The design favours proven, low-overhead building blocks that suit an ad-hoc analytical tool.

DECISION	CHOICE	WHY
Orchestration	Step Functions with a Map state	Built-in iteration, retries, and error handling; a proven pattern
Original PDF source	S3 (existing output bucket)	The rendered original is already stored there by the current system
Sent PDF source	Microsoft Graph API (Outlook search)	The sent email is the source of truth for what the customer actually received

DECISION	CHOICE	WHY
Comparison engine	AWS Bedrock (Claude)	Compares documents textually and visually, and can describe nuanced differences
Results storage	S3 JSON, date-partitioned	Directly queryable via Athena; no database or ETL needed
Prompt management	SSM Parameter Store, versioned	The comparison prompt can be refined without redeploying; each finding records the version used
Operating model	Developer-operated, ad-hoc batches	Not a production system; monthly runs with manual analysis and reporting

The comparison itself

For each contract note where both PDFs are in hand, Bedrock is given the two documents and a prompt asking whether they are identical and, if not, to describe each difference with its location and severity. The response is stored as a structured finding: a match/mismatch verdict, a list of differences, and a one-line summary.

Getting this right is mostly a prompt-engineering exercise. The prompt lives in Parameter Store rather than in code so it can be tuned quickly, and each finding records which prompt version produced it. The estimate explicitly budgets for several iteration cycles, running a batch, reviewing the signal-to-noise with BRYT, and refining, until the output reliably flags meaningful edits without drowning them in noise.

What a finding records

Each finding is one JSON record per contract note, written to S3 and queryable in Athena.

FIELD	HOLDS
Offer reference / document ID	Which contract note this finding is for
Run ID & timestamp	Which batch produced it and when
Status	match, mismatch, or error
Original & sent PDF keys	Where each source document was found (sent key only if located)
Comparison result	Bedrock's verdict: identical flag, list of differences, summary
Prompt version	Which prompt version produced the result (for traceability)
Error message	Populated when a step could not complete

How the edge cases are handled

SITUATION	BEHAVIOUR
Original PDF not in S3	Record 'error: original not found'; continue to next record
No matching sent email	Record 'error: email not found'; continue

SITUATION	BEHAVIOUR
Multiple matching emails	Use the most recent; log a warning
Email has no PDF attachment	Record an error finding; continue
Bedrock call fails	Retry once, then record 'error: comparison failed'; continue
Bedrock reply unparseable	Store the raw response and flag it for manual review

Open questions

This estimate has a hard external dependency on Microsoft 365 access. These need BRYT confirmation before work can complete.

#	QUESTION	WHY IT MATTERS
9	Can BRYT provide an Azure AD app registration with Mail.Read (or Mail.ReadBasic) for the Graph API?	This is the critical dependency, without it the sent PDFs cannot be retrieved and the tool cannot function.
10	Which mailbox(es) are contract notes sent from?	Determines which mailbox the search runs against, could be one shared mailbox or several users.
11	How do we correlate a sent email to a specific contract note?	Assumption: the subject line or attachment filename contains the offer reference (e.g. OP-56412). Needs confirmation.

Critical dependency

Question 9 is a blocker rather than a detail. The whole tool depends on read access to the mailbox where contract notes are sent, which requires BRYT's M365 admin to grant the permission. It is worth securing this early, as everything else is downstream of it.

Delivery breakdown

Estimate 5 is ~12.4 days. Unlike the others, a large share is prompt iteration and analysis rather than pure build.

AREA	SCOPE
Infrastructure	Step Function, the six Lambdas, results S3 bucket, Secrets Manager, SSM prompt parameter, IAM, Athena/Glue table
Graph API integration	OAuth client-credentials auth, mailbox search, PDF attachment extraction
Bedrock integration	Initial comparison prompt, model invocation, structured response parsing

AREA	SCOPE
Prompt iteration	3-5 cycles of running batches, reviewing output with BRYT, and refining the prompt
Analysis & reporting	Athena queries for common patterns, producing the spreadsheet report for BRYT
Buffer	Graph API limitations and Bedrock output tuning

A note on effort

This estimate is less predictable than the others because its value depends on comparison quality, which is reached through iteration rather than a fixed build. The figure includes that iteration and a buffer for the Graph API and Bedrock tuning; the underlying spec notes a wider 13-20 day range depending on how many refinement cycles BRYT wants.